

Runtime Polymorphism using Inheritance and Method Overriding

Java Programming Laboratory Record — Employee Salary Details

1. Objective / Aim

To write a Java program that demonstrates **runtime polymorphism (dynamic method dispatch)** by defining a super class `Employee` and two sub classes `RegularEmployee` and `ContractEmployee` that override the method `salaryDetails()`. The program accepts an `Employee Id` from the user and, depending on the first character of the `Id`, computes and displays the appropriate salary break-up using a super class reference that points to a sub class object.

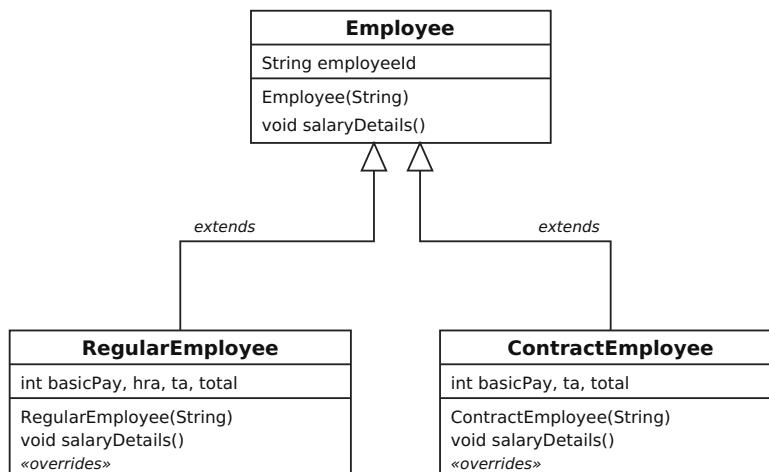
2. System Requirements

Hardware	Standard PC setup — Intel Core i3 / i5 processor or higher, 4 GB / 8 GB RAM, 256 GB storage, standard keyboard and monitor.
Operating System	Windows 10 / 11, Linux (Ubuntu) or macOS.
Software	Java Development Kit (JDK 17 or JDK 21).
Editor / IDE	Eclipse, IntelliJ IDEA, NetBeans or VS Code (any plain text editor with a terminal is sufficient).

3. Algorithm / Logic Description

- Start.**
- Create a super class `Employee` with a `String` data member `employeeId`, a parameterised constructor that initialises it using `this`, and a method `salaryDetails()` that prints a default message.
- Create a sub class `RegularEmployee` that extends `Employee`; call `super(employeeId)` from its constructor and **override** `salaryDetails()` to compute `Total = Basic Pay (25000) + HRA (15000) + T.A (5000)`.
- Create a second sub class `ContractEmployee` that extends `Employee`; call `super(employeeId)` and **override** `salaryDetails()` to compute `Total = Basic Pay (12000) + T.A (3000)`.
- In the `main()` method of the public class `EmployeeDemo`, create a `Scanner` object and read the `Employee Id` as a string.
- Declare a super class reference variable `Employee emp`;
- Examine the first character of the `Id` using `id.charAt(0)`:
 - if it is `'R'` or `'r'` → `emp = new RegularEmployee(id)`;
 - else if it is `'C'` or `'c'` → `emp = new ContractEmployee(id)`;
 - otherwise display *Invalid Employee Id*, close the `Scanner` and terminate.
- Invoke `emp.salaryDetails()`. The JVM decides **at run time** which overridden version to execute, based on the actual object referred to — this is runtime polymorphism.
- Close the `Scanner` object to release the input stream.
- Stop.**

4. UML Class Diagram



The hollow triangular arrow head pointing towards `Employee` denotes generalisation (the *extends* relationship). The class `EmployeeDemo` is the driver class containing `main()`; it only uses these classes.

5. Source Code

File name: **EmployeeDemo.java** (saved inside the package folder **Excercise**)

```

package Excercise;

import java.util.Scanner;

// Super class
class Employee {
    String employeeId;

    Employee(String employeeId) {
        this.employeeId = employeeId;          // 'this' refers to current object
    }

    void salaryDetails() {
        System.out.println("Salary details not available");
    }
}

// Sub class 1 - inherits Employee
class RegularEmployee extends Employee {

    RegularEmployee(String employeeId) {
        super(employeeId);                    // calls super class constructor
    }

    @Override                                // method overriding
    void salaryDetails() {
        int basicPay = 25000;
        int hra = 15000;
        int ta = 5000;
        int total = basicPay + hra + ta;
        System.out.println("Salary Details:");
        System.out.println("Basic Pay: " + basicPay + " HRA: " + hra
            + " T.A: " + ta + " Total Amount: " + total);
    }
}

// Sub class 2 - inherits Employee
class ContractEmployee extends Employee {

    ContractEmployee(String employeeId) {
        super(employeeId);
    }

    @Override                                // method overriding
    void salaryDetails() {
        int basicPay = 12000;
        int ta = 3000;
        int total = basicPay + ta;
        System.out.println("Salary Details:");
        System.out.println("Basic Pay: " + basicPay + " T.A: " + ta
            + " Total Amount: " + total);
    }
}

// Driver class
public class EmployeeDemo {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        System.out.print("Enter Employee Id: ");
        String id = sc.nextLine();

        Employee emp;                        // super class reference

        // R = Regular Employee
        // C = Contract Employee
        if (id.charAt(0) == 'R' || id.charAt(0) == 'r') {
            emp = new RegularEmployee(id);
        }
        else if (id.charAt(0) == 'C' || id.charAt(0) == 'c') {
            emp = new ContractEmployee(id);
        }
        else {
            System.out.println("Invalid Employee Id");
            sc.close();
            return;
        }

        emp.salaryDetails();                  // Runtime polymorphism
        sc.close();
    }
}

```

6. Compilation & Execution Commands

Save the file with the same name as the public class, i.e. **EmployeeDemo.java**, inside a folder named **Excercise**. Open the terminal in the *parent* folder of Excercise and type:

```

// Compile the source file
javac Excercise/EmployeeDemo.java

// Run the compiled bytecode using the fully qualified class name
java Excercise.EmployeeDemo

```

If the package `Excercise;` statement is removed, the simpler commands `javac EmployeeDemo.java` and `java EmployeeDemo` may be used instead.

7. Output Screen

Case 1 — Regular Employee

```
C:\Java> java Excercise.EmployeDemo
Enter Employee Id: R101
Salary Details:
Basic Pay: 25000 HRA: 15000 T.A: 5000 Total Amount: 45000
```

Case 2 — Contract Employee

```
C:\Java> java Excercise.EmployeDemo
Enter Employee Id: C202
Salary Details:
Basic Pay: 12000 T.A: 3000 Total Amount: 15000
```

Case 3 — Invalid Employee Id

```
C:\Java> java Excercise.EmployeDemo
Enter Employee Id: X999
Invalid Employee Id
```

8. Result

The Java program demonstrating runtime polymorphism through inheritance and method overriding was written, compiled and executed successfully, and the output was verified for Regular, Contract and invalid Employee Ids.